Journal of Systems Architecture 156 (2024) 103266



Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc



Toward Linux-based safety-critical systems—Execution time variability analysis of Linux system calls

Markel Galarraga [a,b,*], Charles-Alexis Lefebvre [a], Jon Perez-Cerrolaza [a], Jose A. Pascual [b]

[a] Ikerlan Technology Research Centre, Basque Research and Technology Alliance (BRTA), Arrasate/Mondragon, Spain
[b] Faculty of Informatics, University of the Basque Country (UPV/EHU), Donostia-San Sebastián, Spain

ARTICLE INFO

Keywords:
Linux
Execution time
System calls
Real-time
Safety-critical systems

ABSTRACT

Modern transportation and industrial domain safety-critical applications, such as autonomous vehicles and collaborative robots, exhibit a combination of escalating software complexity and the need to integrate diverse software stacks and machine learning algorithms, consequently demanding complex high-performance hardware. Linux's extensive platform support and library ecosystem make it a valuable general-purpose operating system for developing complex software systems. However, because the Linux kernel has not been designed to comply with safety standards, it has a high execution path variability and does not provide execution time guarantees. In this context, several research initiatives have studied the usage of Linux for developing complex safety-related systems, focusing on topics that include its development process, isolation architectures, or test coverage estimation. Nonetheless, execution-time analysis and providing temporal guarantees is still a challenge. This work extends the novel statistical analysis of Linux system call execution paths with the analysis of execution-time variability and proposes a method for estimating the worst-case execution time, forming a sound approach for an in-depth analysis of the Linux kernel execution paths and execution times for safety-related systems. The proposed method is applied to a representative use case that implements an Autonomous Emergency Brake application in an NVIDIA Jetson Nano board connected to the CARLA autonomous driving simulator.

1. Introduction

Safety-critical embedded systems are systems whose failure can lead to catastrophic consequences (e.g., human casualties in autonomous car accidents). They are programmable systems composed of electronics and software, and they need to be developed by adhering to strict safety certification standards. Examples of such standards are the generic IEC 61508 or the automotive ISO 26262. In the past few decades, diverse industry sectors have invested substantially in novel safety-oriented systems that have the potential to bring about revolutionary changes across multiple market domains, particularly within the realm of functional safety. Many of these pioneering safety-related systems exhibit notable characteristics, including high performance requirements, escalating software complexity, and the incorporation of open-source components, along with Machine Learning (ML) algorithms and associated software stacks. Autonomous cars and automatic train operation are some representative examples of such systems. An Operating System (OS) can effectively accommodate the integration of these complex applications with the required software stacks while simultaneously offering comprehensive support for functional safety. As a result, this could facilitate the usage of complex applications in compliance with functional safety standards [1]. Linux is the leading OS from embedded systems to supercomputers and almost everywhere in between [2]. In addition, Linux has already been deployed in critical applications (e.g., telecommunication, banking) and dependable systems such as spacecraft (e.g., SpaceX Falcon 9, Dragon) [3]. Therefore, there is great interest in making its usage in safety-critical systems possible, since it would greatly help reduce development efforts and costs. Multiple works have been carried out to join safety certification requirements and technological advancements, mainly focusing on Linux [4–7]. Moreover, Linux's open-source development model has been argued by the SIL2LinuxMP project as possibly valid for IEC 61508 certification via Route 3_S, also known as "compliant non-compliant development" [5].

Likewise, the doctoral thesis Statistical Path Coverage for Non-Deterministic Complex Safety-Related Software Testing (SPC) [6] proposes statistical methods to overcome the impossibility of achieving

* Corresponding author at: Ikerlan Technology Research Centre, Basque Research and Technology Alliance (BRTA), Arrasate/Mondragon, Spain.
E-mail addresses: mgalarraga@ikerlan.es (M. Galarraga), calefebvre@ikerlan.es (C.-A. Lefebvre), jmperez@ikerlan.es (J. Perez-Cerrolaza), joseantonio.pascual@ehu.eus (J.A. Pascual).

https://doi.org/10.1016/j.sysarc.2024.103266
Received 1 December 2023; Received in revised form 11 July 2024; Accepted 24 August 2024
Available online 28 August 2024
1383-7621/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

100% test coverage of Linux system call execution paths. That work focuses on the test coverage of every possible execution path that the Linux system calls can take. The execution path taken depends on the state of the system at that point in time. This state depends on multiple and complex elements of the system and cannot be controlled or predicted [6]. Therefore, obtaining a 100% test coverage of system call execution paths is unfeasible. Consequently, SPC focuses on extracting system call execution paths (traces) in a testing process, and statistically estimating the test coverage and the risk of untested paths appearing when in operation. However, SPC obviates execution-time behavior and solely focuses on execution path coverage. Timing analysis is also essential for safety-critical systems, since they must meet hard real-time requirements. For example, an Autonomous Emergency Brake (AEB) must anticipate the risk of collision within bounded time windows to activate the brakes and slow down the vehicle, avoiding or minimizing the impact. If the activation of the brakes is delayed, the risk of collision is higher. Hence, timing constraints are as crucial as functional constraints.

In this work, we extend SPC by finding a relationship between system call execution paths and their execution times and by explaining how said relationship can be used to enhance the execution-time variability analysis. To do that, we design a representative use case consisting of an AEB implemented with YOLOv3 and run it in an Nvidia Jetson Nano connected to a vehicle in the CARLA simulator. YOLOv3 is a very fast object detection model based on a single neural network [8]. We also implement a simpler second use case that does not use the GPU and TCP/IP communication, as the first does, and just executes a few system calls. This use case allows us to obtain results that are easier to work with but help us understand the general behavior. Both use cases are explained in more detail in Section 5. In addition to implementing the use cases, we extend the capabilities of the tools used in SPC and give them the ability to extract execution times together with traces. Next, we show the link between traces and execution times and explain how it could be exploited with probabilistic methods. Finally, we experimentally use the mentioned probabilistic methods to analyze execution times with Probabilistic Worst Case Execution Time (pWCET), a probabilistic methodology that can be used to estimate worst-case execution times when static analysis is not possible. To the best of our knowledge, neither SPC nor other work has analyzed Linux system call execution time variability in combination with system call path analysis in the context of safety-critical systems.

The goal of our work is to add execution times to the SPC methodology (which are disregarded by it), study the relationship between execution paths of system calls and their execution times, and propose a way to take advantage of said relationship, to study the possibility of using Linux in safety-critical systems.

1.1. Contributions

The summary of the contributions of this work is:

- We modify the tool used by SPC (DB4SIL2) to make it able to collect execution times together with system call traces and make the modifications available online.
- We study the relationship between system call execution paths and their execution time and find that each path (unique trace) has its own execution-time distribution.
- We propose to use pWCET analysis for each unique trace instead of using it with all the results of the system call. This allows us to obtain shorter execution time estimates. We carry out a first approach with the most common execution path of a system call. However, we identify some shortcomings in our use of pWCET that need to be resolved in future work.
- We explain how we intend to utilize the pWCET estimates with a runtime monitor that detects if unique traces with too long executing times are being executed. The monitor relies on the fact that each unique trace has its own execution time distribution and not necessarily on the pWCET estimates. Other methods could be used to model the execution time distributions of unique traces.

1.2. Structure of the paper

The rest of this paper is organized as follows: Section 2 presents other works that have also focused on the usage of Linux in safety-critical systems, and works that have studied the Worst Case Execution Time (WCET) of Linux systems. Section 3 introduces works and concepts that serve as the basis for our work. In Section 4 the methodology followed in this work is explained. Section 5 presents our use cases and the experimental set-up. Section 6 introduces the results and the discussion. Finally, Section 7 concludes the paper and presents some lines of future work.

2. Related work

Multiple works have studied the usage of Linux in safety-critical systems. The doctoral dissertation Statistical Path Coverage (SPC) [3,6,9,10] statistically analyzes the Linux kernel execution paths and presents a methodology to estimate the system call execution path test coverage obtained in a testing process and the risk of executing untested paths. The SIL2LinuxMP project [5] "aims at the certification of the base components of an embedded GNU/Linux RTOS[1] running on a single-core or multi-core industrial COTS[2] computer board." [11]. Various works [12–16] serve as preliminary and as the base for the SIL2LinuxMP project by analyzing the Linux kernel's execution path and execution time variability. Previous to SIL2LinuxMP, the SIL4Linux project [17] studied the possibility of Linux satisfying SIL 4 in some restrictive conditions, by designing a system with tracing and profiling tools, formal methods, and a database management system. SIL 4 is the highest safety integrity level in the IEC 61508 standard and refers to the highest relative level of risk reduction provided by the protection layers on a safety-critical system.

Chen et al. show in their study [18] the differences in observed traces in two different scenarios: they compare the effect of different system loads in the first, and they compare different file systems in the second. Enabling Linux In Safety Applications (ELISA) and Automotive Grade Linux (AGL) are two other projects that aim to use Linux in safety-critical environments. The former is focused on "defining and maintaining a common set of elements, processes, and tools that can be incorporated into Linux-based, safety-critical systems amenable to safety certification." [4]. The latter's objective is to "bring together automakers, suppliers, and technology companies for the purpose of building Linux-based, open-source software platforms for automotive applications that can serve as de facto industry standards. AGL addresses all software in the vehicle: infotainment, instrument cluster, heads-up-display (HUD), telematics, connected car, advanced driver assistance systems (ADAS), functional safety, and autonomous driving." [19].

Procopio et al. [20] claim that if projects such as SIL2LinuxMP are focused not only on safety but also on security, the interest of the open-source community for safety-critical systems will increase. Craveiro et al. [21] create a modified and minimal Linux kernel to integrate into systems that must comply with the aeronautical ARINC 653 specification. Different approaches have also been followed [22], in which the authors show a methodology to find sources of *Software Aging* in the Linux kernel, which refers to the tendency of systems to show degrading performance and eventually fail due to error conditions that accumulate over time, such as round-off errors, unreleased memory regions, or storage space fragmentation. In their analysis, they find that the Linux kernel has bugs that manifest as a statistically significant aging trend of memory consumption. Finally, Cinque et al. [23] introduce the concept of real-time containers as a solution for temporal and fault isolation in mixed-criticality architectures. They offer a reference architecture and

[1] Real-Time Operating System.
[2] Commercial-Off-The-Shelf.

initial prototype implementation using Docker containers over a Linux kernel patched with PREEMPT_RT. Although our work focuses only on Linux, the underlying hardware must also be appropriate for safety-critical systems in a real use case because certification is obtained for the entire system, not for its individual components. Perez-Cerrolaza et al. [7,24] present an overview of the current state of affairs in that regard.

Regarding works that have studied the Worst Case Execution Time (WCET) of Linux systems, de Oliveira et al. [25,26] model the real-time Linux kernel (patched with PREEMPT_RT) and define a set of properties and rules regarding its timing behavior, for example rule 13 "(R13): Calling the scheduler always results in a context switch". Then, they execute an analysis that outputs a theoretical bound that considers the sources of delays, and they present the tool they developed for the task. The research conducted by Silva et al. [27] presents a Probabilistic Worst Case Execution Time (pWCET) analysis of a bubble sorting application run in an Ubuntu microcomputer and concludes that Extreme Value Theory (EVT) is adequate for estimating WCETs in complex systems. Other works have also studied pWCET in complex systems – those in which classic static methods are unfeasible – and are overviewed in a survey by Cazorla et al. [28], concluding that probabilistic methods have been extensively investigated in the surveyed period, most of them using EVT.

Within the context of using Linux in safety-related systems, we propose to study the time taken by the kernel's system calls for each of their observed traces. We plan to associate the different execution paths traversed by our use cases and their corresponding execution times measured during the observation period. This will be used as the base for online monitoring and to check that, at run time, the system traverses the execution paths that have already been seen and whose worst-case execution times are estimated beforehand. Otherwise, the monitor could bring the system to a safe state. This paper focuses on a methodology for associating execution paths and their corresponding execution times before operational use. As far as we know, the relationship between Linux system calls and trace execution times has not been studied in this precise context. Okech et al. [15] present some results showing the time distribution of a system call, but do neither analyze it nor give any conclusions in that regard. The work from Finney [29] does examine system call timings and justifies the usage of Linux for millisecond order deadlines. However, they use a single-core system with a 2.2.12 Linux kernel, and therefore their results no longer hold for the complex multicore systems of today.

3. Preliminaries

3.1. Terminology

Throughout this work, we adopt the terminology used in SPC [6]. We summarize here the principal terms.

- System call (or syscall): A function of the Linux kernel that serves as the interface between kernel and user space.
- Trace: The execution path a system call follows, i.e. the specific control-flow path of execution it takes. System calls can follow different execution paths; thus, multiple different traces can be generated by the same system call.
- Unique trace: A particular path a system call has taken. For example, a system call executed ten times can generate unique trace *A* eight times and unique trace *B* two times.
- Most common trace (MCT): The unique trace that is most frequently traversed in the execution of a system call. As explained in SPC [9], system calls tend to follow certain paths most of the time, while following other paths much less. The former can be considered the most common traces of the syscall, while the latter can be considered rare traces.
- Test campaign: An iteration of the testing process where the tested application is executed a number of times.

3.2. Safety-critical systems and Linux

In broad terms, functional safety is a risk management methodology applied to electrical, electronic, and programmable electronic systems. Its purpose is to ensure the integrity of processes that involve potential risks of significant magnitude that could lead to accidents with major implications. In other words, it aims to prevent unacceptable risks that may directly or indirectly result in physical harm or damage to people's health. Functional safety can also be defined as the "absence of catastrophic consequences on the user(s) and the environment" [30]. IEC 61508 [31] is a generic functional safety standard used as a reference safety standard by other domain-specific standards such as ISO 26262 for automotive and EN 50126 for railways [7,24]. These standards consider two basic types of errors: random hardware errors (e.g., memory bit-flip) and systematic errors introduced in the development process (e.g., design errors).

Linux is a highly complex product, and its certification for safety-critical applications is not straightforward. Works focusing on using Linux in safety-critical systems do not intend to create a definitive "safe" version that suits all use cases [5]. Instead, the goal is to demonstrate that, thanks to the Linux development process and its characteristics, an argument can be made for using Linux in safety-critical systems. Nevertheless, every application, hardware setup, and use case is different, and it is up to the developers to use Linux in a safe and sound way and justify the decisions in terms of safety to make certification of their system possible. It is essential to emphasize that certifying a system that uses Linux for functional safety does not automatically certify Linux itself as a product. Each system is unique, and certification is obtained for the complete system, not individually for its components. Furthermore, Linux constantly undergoes changes and cannot be considered a single final product but an evolving one. It is worth mentioning that projects that focus on the usage of Linux for safety-critical systems, such as SIL2LinuxMP or ELISA, are generally not concerned only with the kernel but also with libraries, tools, and everything that makes Linux an operating system.

3.3. Statistical path coverage

Linux may be initially dismissed for safety-critical applications due to its difficult-to-predict nature caused by its high execution path variability. However, the doctoral thesis titled "Statistical Path Coverage for Non-Deterministic Complex Safety-related Software Testing" (SPC) [6] proposes a statistical approach as an alternative to classic methods.

SPC focuses on Linux system call execution paths. Firstly, it rules out static analysis as a possibility. Static analysis involves analyzing every possible execution path of a program. It is a common approach in programs that can take different execution paths but in which forcibly controlling the decision expression that governs them is overly difficult. In other words, in static analysis, all execution paths are analyzed individually, to determine if they are all suitable for use. However, this is not feasible for Linux because of the exceedingly large number of possibilities on its system calls, which are in general not extremely specialized, but instead do "one thing" under a large number of situations and ramify in potentially many execution paths (as a consequence of Linux being general-purpose). Moreover, indirect function calls in the Linux kernel are prevalent [9], and static analysis of these calls is generally considered untractable. Indirect calls are function pointers that select the invocation of a specific implementation at run time.

Secondly, as an alternative to static analysis, SPC proposes a dynamic analysis of kernel traces by tracing the execution paths of system calls. This involves running the application repeatedly and collecting kernel traces. The work uses the concept of unique traces, which identify specific execution paths of system calls. Unique traces, therefore, serve to identify each possible different execution path that has appeared throughout the application runs. What is observed through

this analysis is that, although new unique traces continue to appear initially, the number of new unique traces that appear decreases as the number of executions increases. This implies that, although the number of theoretical possibilities may be very large, only a certain number of possibilities are seen in practice. Based on that information, the project proposes statistical methods to estimate the test coverage of possible unique traces and the risk of system calls traversing an untested path at run time.

3.4. WCET and pWCET

The Worst Case Execution Time (WCET) of a computer program is the longest amount of time it takes to execute. In real-time systems, it is an important metric that aims to ascertain that response time thresholds will not be surpassed. There are two main ways to determine WCET: static methods and measurement-based methods [32]. Static methods seek absolute theoretical rigor and strictly depend on the accurate description of the timing behavior of the hardware and software internals. This generally produces pessimistic results that are exacerbated as the system becomes more complex. Measurement-based methods, on the other hand, execute the given program with a subset of possible states or values of all the features that affect execution times. If the program were to be executed with the values that produce the worst-case execution time, or if all possibilities were executed, a safe upper bound would be obtained. Those specific values are generally unknown and uncontrollable, and executing every possibility normally proves unfeasible. Therefore, only estimates or distributions can be obtained.

The increasing complexity of current hardware and software makes both types of methods difficult to use, and probabilistic reasoning aims to mitigate their limitations [7,24,28]. Probabilistic analysis methods sample execution-time observations according to given criteria, and then fit the results to a probability distribution, thus obtaining the pWCET estimation [28].

Probabilistic analysis methods have been used with Linux systems before [27], but to the best of our knowledge, they have never been used focusing on Linux system call traces to study the impact that execution path variability has on execution times. In our work, we use EVT as a tool to estimate the pWCET of Linux system calls. EVT is used to estimate the probability of extreme events, which in our case are the WCETs of system calls. We also use EVT to assess that the size and number of campaigns are statistically significant, as done in SPC, by using it with the number of unique traces found per campaign. Finally, there is a requirement to EVT that data be independent and identically distributed (i.i.d.). We test this by using autocorrelation and the Kolmogorov–Smirnov and Ljung–Box tests.

4. Methodology

The first step of our method is repeatedly executing our programs and reading the kernel traces they generate. We use two different use cases. The first use case, UC1, involves a simulation where a crash is avoided by using an Autonomous Emergency Brake (AEB) implemented in the CARLA simulator. The simulation lasts about 30 s, and traces for the whole execution are collected for analysis. Each run of the application generates a 1.5 GB trace file. One run of the use case includes around 15000 calls to the `SyS_ioctl` system call. The second use case, UC2, is a very simple program that, only executes the `SyS_openat` system call twice per run. More details about both use cases can be found in Section 5. Due to the difference in number of system call usage for both use cases, UC1 needs fewer runs to yield statistically significant results, while UC2 needs more. Following SPC, we distribute the execution in campaigns. For the first use case (UC1), we choose to run 10 campaigns of 20 executions each. For the second use case (UC2), we execute 100 campaigns of 1000 runs each. These choices are taken based on the validation methodology proposed by SPC, i.e., using EVT to determine that the number and size of campaigns are sufficient by estimating that the number of unique traces found per campaign would remain stable with more campaigns.

In this work, we focus on system calls because they are the interface to the kernel and, therefore, the main way application programs interact with it. Even though more things are happening in the system while our programs are being executed (e.g., kernel housekeeping), we choose to focus on system calls only because this approach allows us to easily check when and why they are executed. In addition, focusing on system calls also enables us to use the tools and methods previously developed for SPC. Extending the analysis to the entire kernel is an equally challenging and interesting continuation of this work.

Some system calls are executed much more often than others for both use cases. Therefore, the number of executions needed to study each system call differs. For example, a single run of UC1 includes around 15000 `SyS_ioctl` calls, while a single run of UC2 calls `SyS_ioctl` just once. In addition, some system calls generate more unique traces than others for both use cases. For that reason, we choose to focus on a single system call for each use case. For UC1, we choose `SyS_ioctl`, because it is used by the program to communicate with the GPU and execute the inference. Therefore, it may be regarded as the most important system call of the application, because it is the one used to detect the collision. For UC2, we choose `SyS_openat` because it is the only system call that generates statistically significant results for posterior analysis due to the simplicity of the use case. The other system calls used by UC2 only yield one or two unique traces per campaign. Despite focusing on those two system calls, we have also seen how the other system calls show similar results, so our conclusions are valid for all of them despite needing more executions to formally prove them.

To collect data, DB4SIL2 [33] is executed in both use cases together with the programs. DB4SIL2 is a tool that can read Linux kernel traces generated by a program run, and extract and process information. In the case of the SPC project, it extracts the system calls the program uses and their traces, and assigns an MD5 hash to every unique trace. The slightest difference in the trace produces a completely different MD5 hash, so only traces that are completely identical will generate the same hash. This hash is then used to investigate trace frequency, identify rare traces, do statistical estimations, etc. DB4SIL2 uses ftrace to obtain kernel traces, specifically the function-graph tracer, which probes both function entry and exit and outputs the trace in a human-readable manner, making the extraction of traces and their analysis easy. In this work, we modify DB4SIL2 to obtain the syscall execution times as well. The DB4SIL2 tool with our modifications can be found in the following repository [34].[3] The function-graph tracer used by DB4SIL2 outputs function execution times, so modifications to ftrace are not necessary. Therefore, only modifications to DB4SIL2 are required, allowing it to read the execution times from the ftrace output when a system call end is detected. The MD5 hash is still calculated without taking the execution time into account, so we end up with the same information as in SPC, but saving execution times together with traces. Therefore, we obtain a collection of pairs `(hash, execution time)` for each execution of the system calls, which are then grouped into campaigns.

Before conducting any analysis, we assess the statistical significance of the dataset using EVT on the `(hash)` component of all pairs `(hash, execution time)`, as done in SPC. Note that, in this case, we are applying the EVT to the number of unique traces found per campaign, not to execution times. In SPC, this is done to assess that the number of campaigns and their size is sufficiently large, and we use it the same way. Applying the EVT requires the sub-dataset to meet the requirement of being i.i.d.. For that reason, we use autocorrelation, Kolmogorov–Smirnov, and Ljung–Box tests, basing ourselves on SPC. In addition, we test the i.i.d. nature of the sub-dataset `(execution time)` because we use EVT to estimate the pWCET with that sub-dataset, and therefore the sub-dataset is required to be i.i.d..

[3] Our data is also available for any interested reader upon request.

After assessing the statistical significance of both sub-datasets (`hash`) and (`execution time`), we extract two metrics: the number of unique traces per campaign and the frequency of these unique traces. By comparing these values with those obtained in the SPC case studies [9], we show that the methodology is suitable for our use cases.

The subsequent phase of our analysis focuses on exploring the relationship between unique traces and execution times. We examine the complete dataset in two distinct approaches for each system call:

- Isolating the sub-dataset (`execution time`): In this approach, we focus solely on the execution times associated with a system call, disregarding distinctions between the execution paths, to study the behavior of the system call as a whole and have a reference with which we can compare the execution time behavior of each unique path.
- Considering the full dataset (`hash, execution time`): Here, we examine the entire dataset, analyzing how the unique traces relate to execution times.

Then, we explain how we intend to exploit the relationship between unique traces and execution times and offer a first approach by using probabilistic methods to estimate the probabilistic WCETs of unique traces. For that, we first study the execution-time distribution curves of the system calls and their unique traces; next, we show how they exhibit appropriate behavior for EVT; and, finally, we use EVT to estimate pWCETs of the most common traces. To choose the models and parameters for the pWCET estimations, we base ourselves on the literature [27,28], and we test different combinations of parameters, showing how different parameters fit different data better. We choose to use the Generalized Extreme Value (GEV) and Gumbel models, and we estimate their parameters by using the Maximum Likelihood Estimation (MLE) and L-moments estimators. We also test different Block Maxima (BM) sizes, which establish the number of samples that form each group or block, and only the maximum value of each block is used to generate the model.

Finally, we show how our approach of estimating the pWCET of unique traces could be used with a runtime monitor in safety-critical systems. The monitor would examine at run time that only tested unique traces are being traversed and would give a warning or stop the system if an untested path is found. In addition, we would know with pWCET which unique traces have execution-time estimates lower than required by the characteristics of the use case. This would allow us to know that if any other unique trace is traversed, there is a danger of surpassing the threshold. Hence, the monitor would also give a warning or stop the system with unique traces that have execution-time estimates higher than required. Since traces are sequences of function calls, the monitor could compare every step of the call chain while the system call is being executed, and hence it could know if a danger exists before the execution of the system call has finished. We explain in Section 7 that this work serves as the base for the runtime monitor and that we intend to work on it in the future.

5. Experimental setup

The main use case we have chosen for our analysis implements an AEB for the CARLA autonomous driving simulator [35]. Specifically, we have developed an application for CARLA in which two vehicles are launched in perpendicular directions near an intersection and start moving toward it. Both cars reach the intersection at the same time, so if none of them brakes, a T-bone crash happens. To prevent the impact, we have developed an AEB that stops one of the vehicles when the impact is about to happen. It is composed of a camera in the CARLA simulator, communication between CARLA and an embedded platform, and the YOLO object detection model running in the embedded platform. In CARLA, we equip one of the cars with a frontal camera that continuously sends frames to an embedded platform that infers whether there is an obstacle in front. The application running on the platform



Fig. 1. Frame from the camera added to the car simulated in CARLA. On the left side of the image, the other car appears, surrounded by a box that indicates that YOLO identifies the object as a car.

receives frames and passes them one by one as input to the YOLO object detection model. YOLO detects objects in an image and tags them as the category to which it believes they belong (e.g., car, tree, traffic light, etc.). If an obstacle is detected by YOLO, the application executes a simple distance calculation algorithm to decide if the distance to it is too short for the current traveling speed of the vehicle, and in that case, a braking order is sent to the CARLA simulator. When the obstacle is no longer there, an accelerating order is sent, and the vehicle with the camera starts moving again. Therefore, when the vehicle equipped with the camera detects that there is another car reaching the intersection, it brakes and lets it pass. When the intersection is clear, it continues on its way. The first frame in which the other vehicle is detected can be seen in Fig. 1. We call this use case UC1.

UC1 is composed of several interacting parts, such as YOLO, communication with CARLA, etc., so we have analyzed it with the strace tool and have seen that the `SyS_ioctl` system call is used by YOLO to communicate with the GPU and make the inference on each frame received from CARLA. That is why, as explained in Section 4, we choose to study the `SyS_ioctl` system call.

To better understand trace behavior, we also add a simpler use case that consists of an application that opens `/dev/random` and `/dev/null`, reads a byte from the former, executes `SyS_ioctl`, writes the byte in the latter, and closes both. The application is based on *randbyte* from [9]. We include the `SyS_ioctl` system call to mimic the behavior of our AEB application. This simple application allows us to examine the kernel behavior without the added complexity of the application. We call this use case UC2.

We execute the AEB (UC1) in an Nvidia Jetson Nano running Linux. The inference to detect the car in front is done with YOLO. CARLA is executed in a Windows PC, and communication between CARLA and the AEB happens via Ethernet. The SIL2LinuxMP architecture is implemented in the Jetson Nano to isolate the AEB, allowing its traces to be unaffected by the rest of the system [5]. The UC2 application is also executed in the Nvidia Jetson Nano, and in this case, there is no communication with the PC. The SIL2LinuxMP isolation architecture is also used. Both use cases are not executed at the same time but separately.

Ftrace, and specifically the function-graph tracer, is used to collect kernel traces and execution times. We configure the tracer activating the `funcgraph-duration` and `graph-time` options. This way, ftrace writes in the trace file the execution time for a function when it returns, including the execution time for the nested functions it called. With these two configurations, we obtain at the return of every system call the total time spent in it. DB4SIL2 uses the "trace" file to read all the traces after each run of the use case has finished. Because a run of UC1 generates many traces (around 1.3 GB), we configure the file's maximum size to be 1.5 GB with the `buffer_size_kb` option. In the

Table 1
Details of the dataset for both use cases.

System call	Total exec.	Total unique traces	Approx. num. of unique traces per campaign
`SyS_ioctl` (UC1)	2,902,072	80,118	17,000
`SyS_openat` (UC2)	200,000	1,017	104

case of UC2, many fewer traces are generated, and the default buffer size of around 1.5 MB is enough. Enabling ftrace, and especially the function-graph tracer, is known to produce overhead in the system. In our case, we are just interested in the behavior across multiple runs of the same program, and since every run is affected by the tracing mechanism, we believe the effect it has in our analysis is minimal.

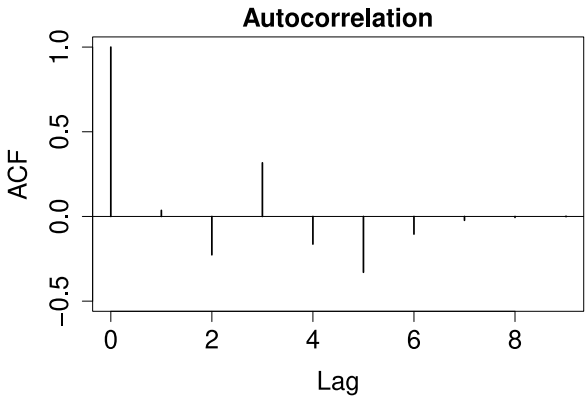
In the Jetson Nano, we use Jetson Linux R32.7.1 from Nvidia, which uses the kernel version 4.9. The only modification we have done to the kernel is enabling ftrace and the function-graph tracer. The CARLA version we use is 0.9.9, without modifications. The virtual camera we add to one of the vehicles has a resolution of 640 × 360 pixels and a field of view of 90°. Finally, we use YOLOv3 [36] for inference.

6. Analysis of the results

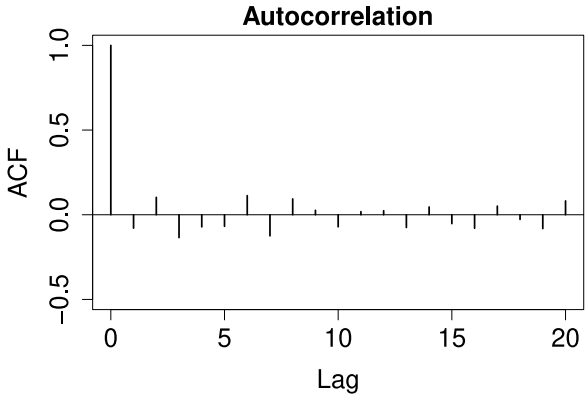
As explained in the previous section, the first step after collecting the data is assessing its statistical significance. Our data has the shape (`hash`, `execution time`), and we must assess the statistical significance of both the hashes and the execution times, the former to verify that our data is similar to that from SPC and the latter to allow using EVT with it.

Table 1 reports details about our dataset. We can see that both use cases differ greatly. For example, the analyzed system call in UC1 is executed many more times than the one analyzed in UC2. This occurs because one run of the application in UC1 needs to execute the system call thousands of times, while in UC2 it is executed only once. Moreover, fewer unique traces are generated in UC2, due to the fact that the application is less complex.

For the sub-dataset (`hash`), we assess its statistical significance by using EVT, as seen in Table 2. We use the GEV model and the MLE estimator. Since data being i.i.d. is a precondition to EVT, we also show the results for autocorrelation, Kolmogorov–Smirnov, and Ljung–Box tests. Autocorrelation is used to check whether the number of unique paths found in a certain campaign has an effect on the number of unique traces found in any other campaign. The Kolmogorov–Smirnov test is used to check if the distribution of unique traces found per campaign is the same in every campaign. The Ljung–Box test is used to test if campaigns are independent of each other by evaluating any-order autocorrelation. It is similar to autocorrelation, but instead of testing every lag separately, it tests the overall independence of the results. With autocorrelation, by feeding it the number of unique traces found per campaign, we see that the number of unique traces found in a campaign is independent of other campaigns, as shown in Fig. 2(a). For UC1, we obtain a maximum correlation value of 0.317, a minimum of −0.329, and a mean value of −0.056. Ljung–Box also indicates independence of data, with a p-value of 0.895. Finally, we take two random samples of half the number of campaigns for the Kolmogorov–Smirnov test, and repeat it multiple times, obtaining p-values in the range [0.3–1]. The p-values for the two tests are far from 0.05, so we can conclude that the number of unique traces found per campaign are independent of other campaigns, and are identically distributed. The results are similar for UC2, as seen in Table 2, although having different numerical results, they also get stable with large campaign numbers. Autocorrelation is shown in Fig. 2(b), with a maximum value of 0.113, a minimum of −0.135, and a mean value of −0.0016. The Ljung–Box test



(a) `SyS_ioctl` system call (UC1).



(b) `SyS_openat` system call (UC2).

Fig. 2. Autocorrelations for the new unique traces found per campaign, sub-dataset (`hash`).

yields a p-value of 0.42, and the Kolmogorov–Smirnov test yields values in the range [0.45–1]. In spite of the good results of the Ljung–Box test, we can see that the autocorrelation results are not optimal. Work must be done in the future to understand the cause for it and strengthen the assessment of the statistical significance of the data.

For the sub-dataset (`execution time`), we test whether the data is i.i.d., to allow the subsequent estimation of pWCETs via EVT. We follow the same procedure as for the (`hash`) sub-dataset. We show in Fig. 3 that autocorrelation for the execution times of the chosen system calls shows good results for both of the use cases, but the Ljung–Box and Kolmogorov–Smirnov tests yield very small p-values (<0.05), therefore indicating that the execution times are neither independent nor identically distributed. Work must be done to understand why this happens and whether it can be solved. For now, we use the pWCET methodology, despite knowing our data is unfit for it, with the goal of showing our approach, and we leave the qualitative analysis of execution times as future work.

Proceeding with the analysis of the new unique traces found per campaign, we display them in Fig. 4. As we can see, the behavior is as expected by SPC: every subsequent campaign yields fewer and fewer new unique traces. The figure shows how UC1 yields many more new unique traces per campaign than UC2, but it also shows that the trend is similar in both cases. This also shows why obtaining 100% test coverage is unfeasible: even with a high number of campaigns, new unique traces keep appearing, and therefore one cannot be sure that every possibility has been observed.

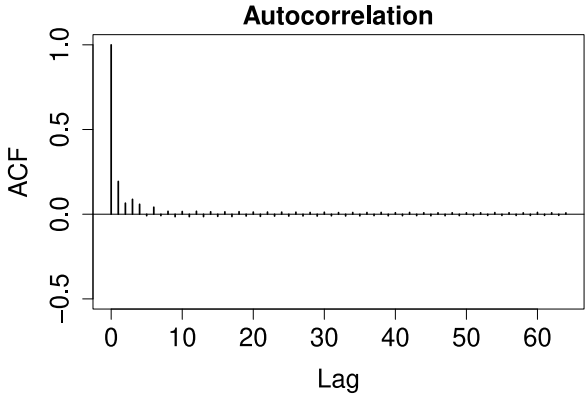
Table 2
Return levels estimated by fitting unique traces per campaign data to the GEV model, using MLE, sub-dataset (`hash`), and their 95% Confidence Intervals (CI).

(a) `SyS_ioctl` system call (UC1).

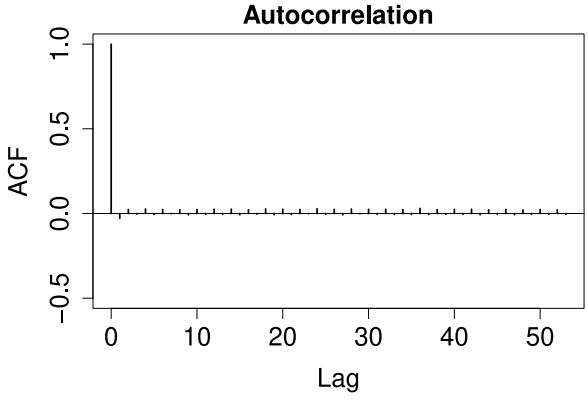
	95% lower CI	Estimate (unique traces)	95% upper CI
10^1-campaign	17,969	18,331	18,694
10^2-campaign	17,979	18,722	19,466
10^3-campaign	17,708	18,902	20,097
10^4-campaign	17,466	18,988	20,509
10^5-campaign	17,299	19,028	20,758

(b) `SyS_openat` system call (UC2).

	95% lower CI	Estimate (unique traces)	95% upper CI
10^1-campaign	116	120	123
10^2-campaign	127	134	141
10^3-campaign	130	143	156
10^4-campaign	131	149	168
10^5-campaign	130	154	177

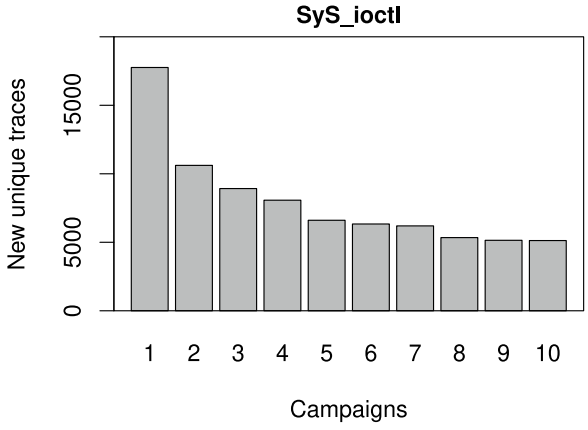


(a) `SyS_ioctl` system call (UC1).

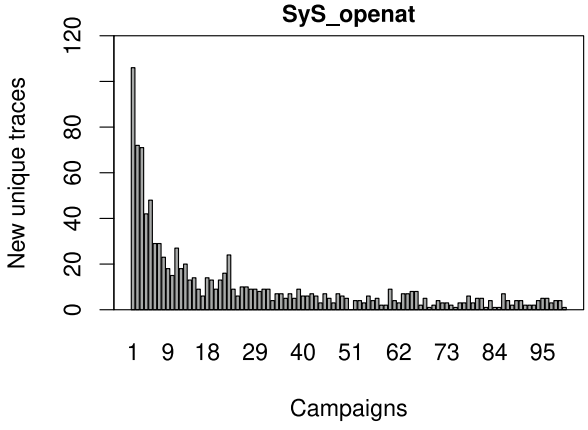


(b) `SyS_openat` system call (UC2).

Fig. 3. Autocorrelations for the sub-dataset (`execution time`).



(a) `SyS_ioctl` system call (UC1).



(b) `SyS_openat` system call (UC2).

Fig. 4. New unique traces found per campaign of executions, sub-dataset (`hash`).

Regarding trace frequency, we can see in Table 3 that the behavior is also the same in this case: a handful of unique traces account for a large percentage of all executions (most common traces), while other unique traces appear very infrequently (rare traces). There is a big difference between use cases in the number of most common traces.

Table 3
Observed frequency of most common traces for each system call, sub-dataset (`hash`).

(a) `SyS_ioctl` system call (UC1).

Trace	Frequency	Accumulated frequency
f92b	0.138	0.138
d0e7	0.114	0.252
10a1	0.102	0.354
bb2e	0.074	0.428
bf15	0.058	0.486
a798	0.043	0.529
4027	0.038	0.567
d09f	0.035	0.602
ea15	0.021	0.623
439a	0.020	0.644

(b) `SyS_openat` system call (UC2).

Trace	Frequency	Accumulated frequency
58f6	0.741	0.741
6289	0.200	0.941
f685	0.001	0.942
25e7	0.000	0.942
1b99	0.000	0.943
6657	0.000	0.943
d849	0.000	0.943
4164	0.000	0.944
95cf	0.000	0.944
921b	0.000	0.944

Table 4
Return levels estimated by the EVT models, (`hash, execution time`) dataset, and their 95% Confidence Intervals (CI).

(a) `SyS_ioctl` system call (UC1).

	95% lower CI	Estimate (us)	95% upper CI
10^1**-test**	589	590	591
10^2**-test**	624	627	630
10^3**-test**	647	653	658
10^4**-test**	663	672	680
10^5**-test**	674	686	697
10^6**-test**	682	696	710
...			
10^{13}**-test**	696	720	743
10^{14}**-test**	696	720	745
10^{15}**-test**	697	721	746

(b) `SyS_openat` system call (UC2).

	95% lower CI	Estimate (us)	95% upper CI
10^1**-test**	273	274	274
10^2**-test**	297	299	300
10^3**-test**	321	323	325
10^4**-test**	345	348	351
10^5**-test**	369	373	376
10^6**-test**	393	397	401
...			
10^{13}**-test**	561	570	578
10^{14}**-test**	585	594	603
10^{15}**-test**	609	619	629

While there are just two traces that account for 94% of the executions of `SyS_openat` in UC2, the 10 most common traces of UC1 account for 64% of the executions, the 20 most common for 75%, and the 944 most common for 90%. This is due to application and system call complexity, while the `SyS_openat` from UC2 just opens files, the `SyS_ioctl` of UC1 is used to communicate with the GPU and execute the inference, so variability is far higher.

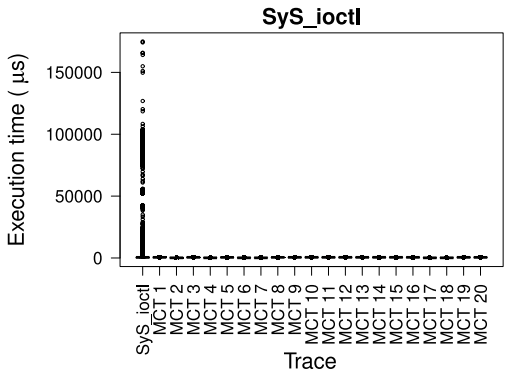
The behavior shown for new unique traces per campaign and for the frequency of traces shows how the SPC methodology is reproducible for our use case. However, going on with the estimation of the test coverage and the risk of finding untested traces, as done in SPC, is outside the scope of this work, so we stop in the validation phase. Instead, we supplement the methodology with the analysis of execution times. Focusing on the entire (`hash, execution time`) dataset, we first show in Fig. 5 for UC1 and in Fig. 6 for UC2, the difference in the execution time distribution of the system calls as a whole and all their unique traces, ordered from most to least common. System call execution times are the grouping of the execution times of all unique traces. We see how every unique trace follows its own execution time distribution. Some traces correspond to the fastest executions of the system call, while others correspond to the slowest executions. However, an interesting trend we observe is that the most common traces never seem to be the slowest, and the fastest one is among them. We can confidently say that there is a strong link between the execution path traversed in each run of a system call and its execution time.

Since system call execution times are groupings of different timing distributions (one for each unique trace), we see in Figs. 7(a) and 7(c) how this affects its density. Instead of having a single curve or bell from the minimum to the maximum times, we find multiple peaks. This complicates the posterior analysis because the behavior is hard to model and therefore probabilistic WCET estimates are unfeasible. However, if we look at the figures with the density plot of individual traces, Figs. 7(b) and 7(d), we can see how the secondary peaks far from the main peak disappear and all values are much closer to the main peak.

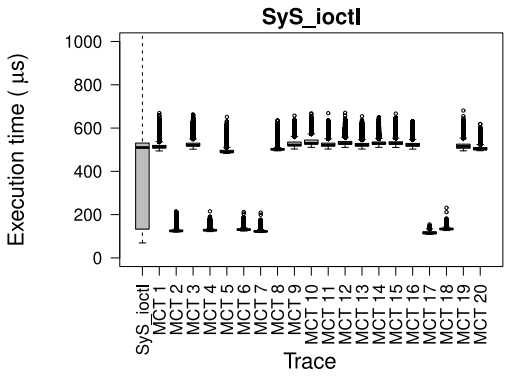
Therefore, we can conclude that analyzing the execution times of each unique trace independently can be beneficial over studying system calls as a whole and disregarding distinct traces, since that proves more difficult and yields worse results. Moreover, some unique traces show maximum times much shorter than the maximum time of the system call. This is especially true for the most common traces. Therefore, their pWCET estimates can be lower than the estimates for the system call as a whole. This could be very useful if there was a way to control which unique traces are executed, or at least to check that only traces with "short" execution times are being executed. We intend to follow this work by designing a runtime system monitor that checks which trace is being executed, therefore ensuring that if an execution path with a long execution time were being taken, the monitor would detect it and trigger the system's safe state to avoid possibly catastrophic consequences. We expand these ideas in Section 7.

As the first approach to those ideas, we show how the WCET of unique traces can be estimated, by doing it with the most common traces of both applications. For that, we use pWCET methodology. The first approaches to fit models to the data show promising results, as shown in Table 4. With parameters estimated with MLE, and for a BM size of 50, we obtain robust return level estimations, even for 10^{15} executions. We use the GEV model for UC1, and the Gumbel model for UC2. It is important to remember that the i.i.d. hypothesis failed with our data and that it is a requirement for pWCET. However, the monitor relies on the fact that each unique trace has its own execution time distribution and not necessarily on the pWCET estimates. Other methods could be used to model the execution time distributions of unique traces (e.g., maximum recorded value, maximum acceptable time, etc.). We leave the study of the causes for the failure of the i.i.d. hypothesis and the study of other methods to model execution times as future work.

We have explored different models (GEV and Gumbel), different estimators (MLE and L-moments), and different BM sizes (from 10 to 16,000) and conclusions about the best fit are unclear. We have found that a different parameter combination for each system call and unique trace makes pWCET estimate values closest to reality and progressively stable as the number of test increases. Many combinations give a good fit of the model to the data, but too large WCET estimates, and vice



(a) Execution time distribution of the `SyS_ioctl` system call and its 20 most common traces (UC1).



(b) Same figure as (a), but with the y-axis is limited at 1000 μs.

Fig. 5. Execution time distributions of the `SyS_ioctl` system call and its most common traces (UC1), (`hash, execution time`) dataset. The figure (b) is identical to (a), but there is a limit on the *y*-axis.

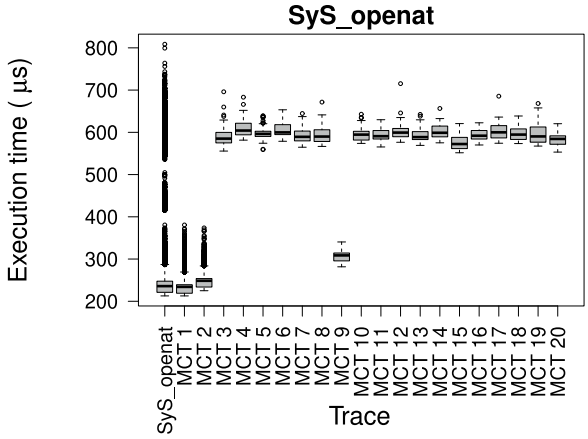


Fig. 6. Execution time distributions of the `SyS_openat` system call and its 20 most common traces (UC2), using the (`hash, execution time`) dataset.

versa. Work must be done to properly choose the model, estimator, and BM size. We also think that it would be interesting to analyze a Linux kernel patched with PREEMPT_RT and compare results.

Another interesting result we see in the figures is that the density of the Most Common Traces (MCTs) for UC1 (Fig. 7(b)) is not completely centered around a single value, despite all values being much closer to the main peak than in the density for the system call (Fig. 7(a)). The density fluctuates around different execution times. To rule out that the fluctuations are caused because of the impact of application complexity (UC1 uses computer vision, GPU, and TCP/IP communication), we show in Fig. 7(d) how UC2 shows the same behavior. These "jumps" make it difficult to fit models to the data, and therefore complicate statistical analysis.

7. Conclusions and future work

In this work, we introduce a Linux system-call execution-time variability analysis that is based on the SPC methodology and joins system call traces with execution times. To do so, we begin with implementing two applications: UC1, consisting of an AEB that runs in an NVIDIA Jetson Nano and is connected to the CARLA autonomous driving simulator; and UC2, that just executes the `SyS_openat`, `SyS_read`, `SyS_ioctl`, `SyS_write`, and `SyS_close` system calls. Then, we modify the tools used in SPC giving them the ability to extract execution times together with the traces. Next, we study the obtained results.

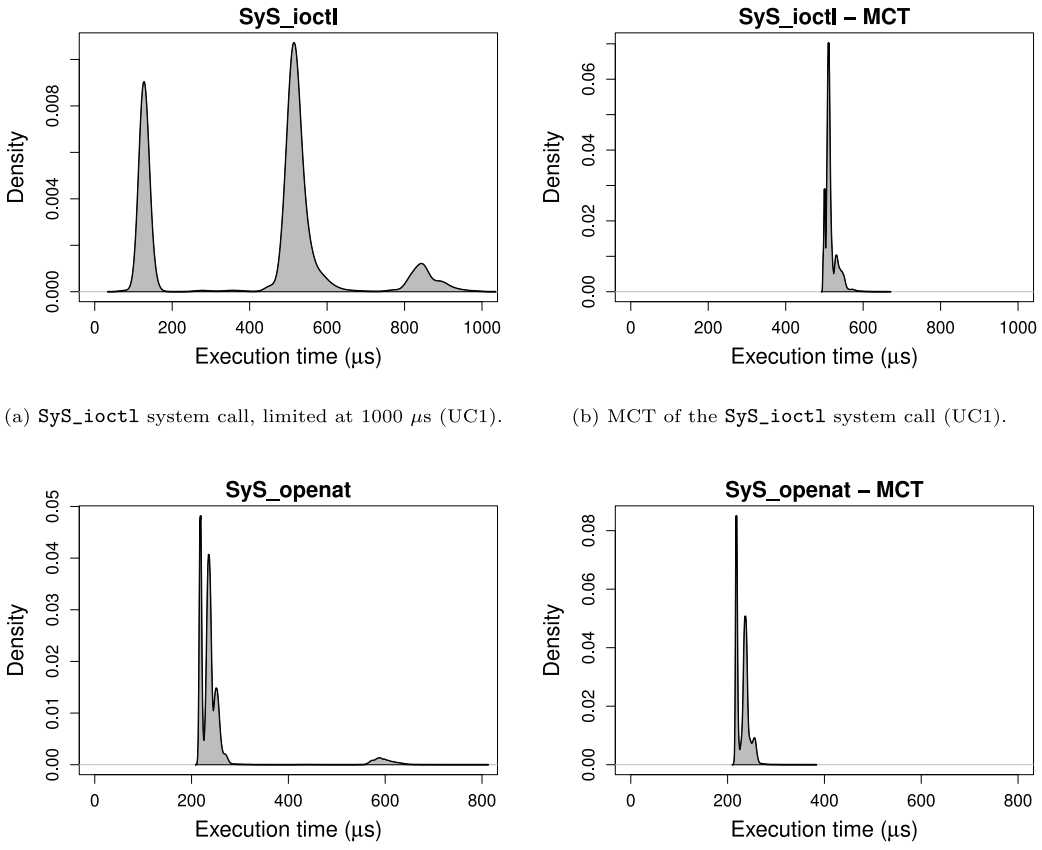
Firstly, we see that the number of new unique traces found per campaign follows the behavior expected in SPC, for both of the use cases, thus evidencing that the SPC methodology can be used in our use case.

Secondly, we show that traces and execution times do have a relationship because the execution time distribution of each unique trace is also unique and bounded more narrowly than the execution time distribution of the system call as a whole. Since each trace has its own execution-time distribution, we could determine which traces are valid or invalid for the threshold of our system, depending on their execution times. This could be exploited if the path taken by the system call could be controlled or monitored. As explained in SPC, the control of system call traces is unfeasible [9], but we believe that syscall monitoring could be used to ensure that only valid paths are executed.

Thirdly, we explain that static methods cannot be used to study WCETs of traces, because, as stated in SPC, the complexity of Linux makes the task unfeasible [9]. Therefore, we carry out a first approach of using the state-of-the-art pWCET analysis with unique traces and show how estimates for unique traces are better than for the system call as a whole. However, we explain how the execution time distributions show "random" peaks and valleys that complicate the analysis. In addition, we also explain that our data fails the i.i.d. test, which is a prerequisite of the statistical tools we use for pWCET analysis, and therefore indicates that our data is unfit for such tools. Still, we choose to carry out the pWCET analysis because it allows us to show one way to exploit the execution time distributions of unique traces.

Looking forward, we intend to repeat the analysis in a real-time Linux system, patched with PREEMPT_RT, and compare the results. We also intend to focus on the underlying hardware and study its impact on our results. We expect to find how both the patch and the hardware affect our results and to exploit it, aiming to improve the pWCET analysis presented in this paper.

Finally, we also intend to use our approach of estimating the pWCET of unique traces to develop a runtime monitor that could be used in Linux-based safety-critical systems. By estimating the pWCET of unique traces, we know which unique traces have execution times shorter than required by the characteristics of the use case. The monitor will check at run time that the system call is traversing one of those unique traces, and not an untested one or one that does not meet requirements. If one of the latter occurs, the monitor will give a warning or stop the system. This will benefit the system in two ways: (1) there will be assurance that only tested unique traces are being traversed, and (2) unique traces that do not meet requirements will be detected before the system call finishes. Because unique traces are chains of function calls, the monitor will check with every call if the execution path is diverting from the expected, and hence detect the danger before the system call has ended.



(a) `SyS_ioctl` system call, limited at 1000 μs (UC1).

(b) MCT of the `SyS_ioctl` system call (UC1).

(c) `SyS_openat` system call (UC2).

(d) MCT of the `SyS_openat` system call (UC2).

Fig. 7. Execution time density for the system calls and their most common traces, (`hash, execution time`) dataset.

CRediT authorship contribution statement

Markel Galarraga: Writing – review & editing, Writing – original draft, Software, Investigation. **Charles-Alexis Lefebvre:** Writing – review & editing, Supervision. **Jon Perez-Cerrolaza:** Writing – review & editing, Supervision. **Jose A. Pascual:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the Spanish Ministry of Economy and Competitiveness MINECO (PID2019-104966GB-I00) and by the Economic Development and Infrastructure Department of the Basque Government (Emaitek program and projects EKK-2023/00012, KK-2023/00090 and Consolidated Groups grant IT1504-22).

References

[1] J. Perez-Cerrolaza, J. Abella, M. Borg, C. Donzella, J. Cerquides, F.J. Cazorla, C. Englund, M. Tauber, G. Nikolakopoulos, J.L. Flores, Artificial intelligence for safety-critical systems in industrial and transportation domains: A survey, ACM Comput. Surv. (2023) http://dx.doi.org/10.1145/3626314.

[2] The Linux Foundation, 2021 Linux foundation annual report. New Horizons for Open Source, 2021, Last accessed on 2023-11-30, https://www.linuxfoundation.org/resources/publications/linux-foundation-annual-report-2021.

[3] I. Allende, N.M. McGuire, J. Perez-Cerrolaza, L.G. Monsalve, J. Petersohn, R. Obermaisser, Statistical test coverage for Linux-based next-generation autonomous safety-related systems, IEEE Access 9 (2021) 106065–106078, http://dx.doi.org/10.1109/ACCESS.2021.3100125.

[4] ELISA - advancing open source safety-critical systems, 2019, https://elisa.tech/Last (Accessed 30 November 2023).

[5] A. Platschek, N. Mc Guire, L. Bulwahn, Certifying Linux: Lessons Learned in three years of SIL2LinuxMP, in: Embedded World, 2018.

[6] I. Allende, Statistical Path Coverage for Non-Deterministic Complex Safety-Related Software Testing, (Ph.D. thesis), University of Siegen, 2021, http://dx.doi.org/10.25819/ubsi/10151.

[7] J. Perez-Cerrolaza, R. Obermaisser, J. Abella, F.J. Cazorla, K. Grüttner, I. Agirre, H. Ahmadian, I. Allende, Multi-core devices for safety-critical systems: a survey, ACM Comput. Surv. 53 (4) (2020) http://dx.doi.org/10.1145/3398665.

[8] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: Unified, real-time object detection, 2016, arXiv arXiv:1506.02640.

[9] I. Allende, N. Mc Guire, J. Perez, L.G. Monsalve, R. Obermaisser, Towards Linux based safety systems—A statistical approach for software execution path coverage, J. Syst. Archit. 116 (2021) http://dx.doi.org/10.1016/j.sysarc.2021.102047.

[10] I. Allende, N.M. Guire, J. Perez, L.G. Monsalve, J. Fernandez, R. Obermaisser, Estimation of Linux kernel execution path uncertainty for safety software test coverage, in: Proceedings -Design, Automation and Test in Europe, DATE, 2021-February, Institute of Electrical and Electronics Engineers Inc., 2021, pp. 1446–1451, http://dx.doi.org/10.23919/DATE51398.2021.9473951.

[11] OSADL, SIL2LinuxMP: OSADL - open source automation development Lab eG, 2014, https://www.osadl.org/SIL2LinuxMP.sil2-linux-project.0.html Last (Accessed 30 November 2023).

[12] N. Mc Guire, P. Okech, G. Schiesser, Analysis of inherent randomness of the Linux kernel, in: Proceedings of the 11th Real-Time Linux Workshop, OSADL, Dresden, 2009, pp. 35–42.

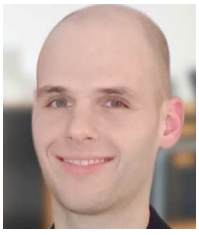
[13] P. Okech, N. Mc Guire, Analysis of Statistical Properties of Inherent Randomness, in: Proceedings of the 12th Real Time Linux Workshop, OSADL, 2010.

[14] P. Okech, N. Mc Guire, W. Okelo-Odongo, Inherent Diversity in Replicated Architectures, in: Fast Abstract and Student Forum Proceedings. the Tenth European Dependable Computing Conference, Paris, France, 2015.

[15] P. Okech, N. Mc Guire, C. Fetzer, W. Okelo-Odongo, Investigating Execution Path Non-determinism in the Linux Kernel, in: Proceedings of the 15th Real Time Linux Workshop, OSADL, Lugano-Manno, Switzerland, 2013.
[16] P. Okech, N. Mc Guire, C. Fetzer, Utilizing inherent diversity in complex software systems, in: Proceedings of the Australian System Safety Conference (ASSC2014), 2014, pp. 71–78.
[17] L. Wang, C. Zhang, Z. Wu, N. Mc Guire, Q. Zhou, SIL4Linux: An attempt to explore Linux satisfying SIL4 in some restrictive conditions, in: Proceedings of the 11th Real-Time Linux Workshop, OSADL, Dresden, 2009, pp. 111–116.
[18] Y. Chen, X. Tang, S. Xu, F. Zhu, Q. Zhou, T.-H. Weng, Analyzing execution path non-determinism of the Linux kernel in different scenarios, Connect. Sci. 35 (2023) http://dx.doi.org/10.1080/09540091.2023.2192442, URL https://www.tandfonline.com/doi/full/10.1080/09540091.2023.2192442.
[19] AGL - Automotive Grade Linux, 2012, https://www.automotivelinux.org/ Last Accessed 30 November 2023.
[20] G. Procopio, Safety and security in GNU/Linux Real Time Operating System Domain, in: Advances in Intelligent Systems and Computing, 925, Springer Verlag, 2020, pp. 245–254, http://dx.doi.org/10.1007/978-3-030-14687-0_22.
[21] J. Craveiro, J. Rufino, C. Almeida, R. Covelo, P. Venda, Embedded Linux in a partitioned architecture for aerospace applications, in: IEEE/ACS International Conference on Computer Systems and Applications, AICCSA, 2009, pp. 132–138, http://dx.doi.org/10.1109/AICCSA.2009.5069315.
[22] D. Cotroneo, R. Natella, R. Pietrantuono, S. Russo, Software aging analysis of the linux operating system, in: Proceedings - International Symposium on Software Reliability Engineering, ISSRE, 2010, pp. 71–80, http://dx.doi.org/10.1109/ISSRE.2010.24.
[23] M. Cinque, D. Cotroneo, Towards lightweight temporal and fault isolation in mixed-criticality systems with real-time containers, in: Proceedings - 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops, DSN-W 2018, Institute of Electrical and Electronics Engineers Inc., 2018, pp. 59–60, http://dx.doi.org/10.1109/DSN-W.2018.00029.
[24] J. Perez-Cerrolaza, J. Abella, L. Kosmidis, A.J. Calderon, F. Cazorla, J.L. Flores, GPU devices for safety-critical systems: A survey, ACM Comput. Surv. 55 (7) (2022) 147:1–147:37, http://dx.doi.org/10.1145/3549526.
[25] D.B. de Oliveira, D. Casini, R.S. de Oliveira, T. Cucinotta, Demystifying the real-time linux scheduling latency, in: Proceedings of the 32nd Euromicro Conference on Real-Time Systems, 165, Schloss Dagstuhl- Leibniz-Zentrum fur Informatik GmbH, Dagstuhl Publishing, 2020, pp. 9:1–9:23, http://dx.doi.org/10.4230/LIPIcs.ECRTS.2020.9.
[26] D.B. de Oliveira, R.S. de Oliveira, Timing analysis of the PREEMPT RT Linux kernel, Softw. - Pract. Exp. 46 (6) (2016) 789–819, http://dx.doi.org/10.1002/spe.2333.
[27] K.P. Silva, L.F. Arcaro, D.B. de Oliveira, R.S. de Oliveira, An empirical study on the adequacy of MBPTA for tasks executed on a complex computer architecture with Linux, in: 2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA), 1, 2018, pp. 321–328, http://dx.doi.org/10.1109/ETFA.2018.8502513.
[28] F.J. Cazorla, L. Kosmidis, E. Mezzetti, C. Hernandez, J. Abella, T. Vardanega, Probabilistic worst-case timing analysis: Taxonomy and comprehensive survey, ACM Comput. Surv. 52 (1) (2019) http://dx.doi.org/10.1145/3301283.
[29] S.A. Finney, Real-time data collection in Linux: A case study, Behav. Res. Methods Instrum. Comput. 33 (2) (2001) 167–173, http://dx.doi.org/10.3758/BF03195362.
[30] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, IEEE Trans. Dependable Secure Comput. 1 (1) (2004) 11–33, http://dx.doi.org/10.1109/TDSC.2004.2.
[31] I.E. Commission, IEC 61508 (1-7): Functional safety of electrical/electronic/programmable electronic safety-related systems (second edition), 2010.
[32] R. Wilhelm, J. Engblom, A. Ermedahl, N. Holsti, S. Thesing, D. Whalley, G. Bernat, C. Ferdinand, R. Heckmann, T. Mitra, F. Mueller, I. Puaut, P. Puschner, J. Staschulat, P. Stenström, The worst-case execution-time problem—overview of methods and survey of tools, ACM Trans. Embed. Comput. Syst. 7 (3) (2008) 36:1–36:53, http://dx.doi.org/10.1145/1347375.1347389.
[33] A. Platschek, DB4SIL2 - Kernel assurance data for SIL2LinuxMP, in: Tech. rep., OSALD, 2016.
[34] DB4sil2 with modifications to read execution times, 2024, https://gitlab.com/markelgalarraga/db4sil2-exec-times Last accessed 27 March 2024.
[35] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, V. Koltun, CARLA: An open urban driving simulator, in: Proceedings of the 1st Annual Conference on Robot Learning, 2017, pp. 1–16.
[36] J. Redmon, A. Farhadi, YOLOv3: An incremental improvement, 2018, arXiv arXiv:1804.02767.



Markel Galarraga received his bachelor's degree in Computer Science and his master's degree in Embedded Systems Engineering from the University of the Basque Country. He is now pursuing a Ph.D. through a collaboration between the University of the Basque Country and the Ikerlan Research Centre. His research interests include the Linux kernel, real-time systems, and safety-related systems.



Charles-Alexis Lefebvre received the Diplôme d'Ingénieur in electronics, M.Sc. in Electronics Engineering, from the engineering school ENSEIRB-MATMECA, Bordeaux INP, France. He has worked as an R&D Embedded Systems Engineer at Ikerlan Research Centre since 2016, where he is leading the research line on Linux for dependable and safety-related systems. Working for different industrial fields for more than 8 years, he has supervised the development of safety-related control embedded platforms for industrial systems in the railway, automotive, and aeronautics sectors.



Jon Perez-Cerrolaza received his Ph.D. degree in computer science from TU Wien. He is currently the technical director of the "electronic embedded systems and cybersecurity" research unit at Ikerlan and was previously Principal Researcher in dependable intelligent systems. His research interests include dependability, machine learning, and cybersecurity technologies. He has worked for more than 15 years in the development and certification of industrial and transportation domain safety-critical systems such as SIL4 onboard railway signaling systems (ERTMS/ETCS). Contact him at jmperez@ikerlan.es.



Dr. Jose A. Pascual obtained his M.Eng. and Ph.D. in Computer Science at the Department of Computer Architecture and Technology of the University of the Basque Country UPV/EHU, Spain. After a period as a postdoctoral researcher at The University of Manchester he is currently a lecturer in operating systems and networks at The University of the Basque Country. His research interests include high performance computing, performance evaluation of parallel systems and real-time and safety-related systems.